Project Overview

LogGuard AI is a cloud-based log anomaly detection system that uses a LogBERT-based model to identify suspicious activity from log data. The system connects multiple AWS services to create a complete pipeline where logs are collected, analysed, stored, visualised, and used to generate automated security alerts.

The project demonstrates how raw log activity can be transformed into useful cybersecurity insights. The system does not only detect anomalies; it also stores the results, sends security email alerts with AI-generated response guidance, and displays the results through a dashboard.

The main stages are:

1. Website hosted on Amazon S3
2. Website event logs generated from user activity
3. Website logs sent through API Gateway and Lambda
4. Website logs stored in CloudWatch
5. LogBERT anomaly detection running on EC2
6. LogBERT anomaly results stored as structured JSON
7. SNS email alerting with OpenAI recommendations
8. Anomaly JSON stored in S3 for dashboard use
9. Streamlit dashboard visualisation

Stage 1: Static Website Hosted on Amazon S3

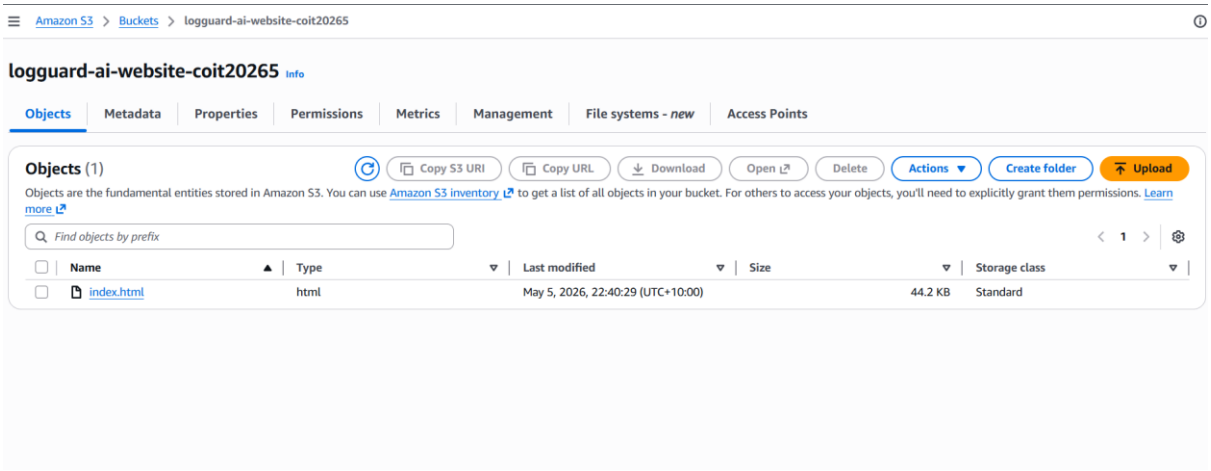


Figure 1: S3 bucket showing the uploaded website file

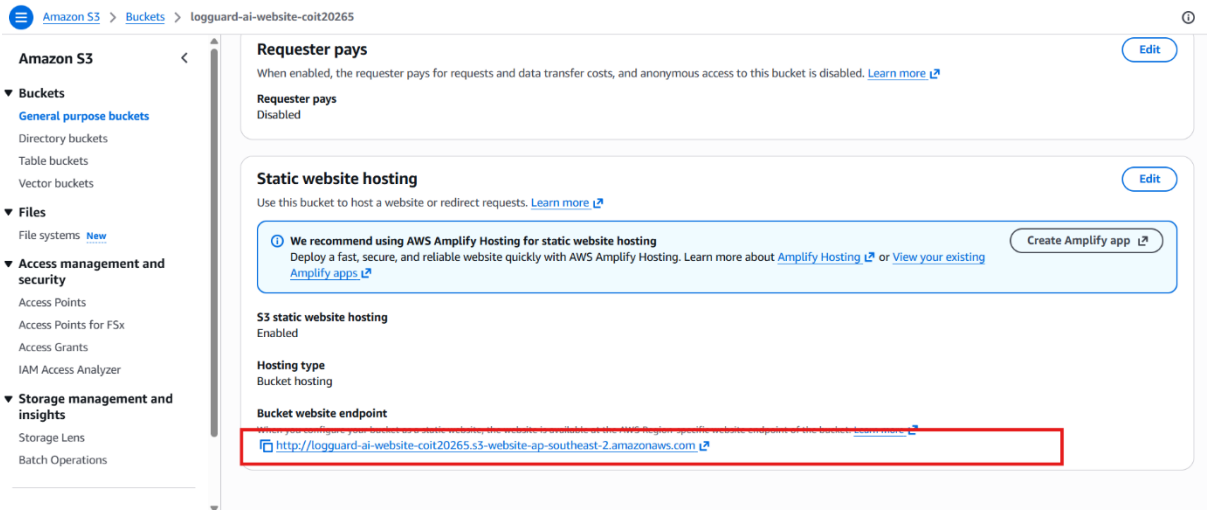


Figure 2: S3 static website hosting endpoint

The first stage of the project is the LogGuard AI website. The website is hosted using Amazon S3 static website hosting. This allows the website to run without a traditional web server, making it lightweight, simple to deploy, and suitable for a cloud-based demonstration.

The website acts as the front-facing part of the project. It introduces the LogGuard AI system, explains the model comparison, shows the project architecture, displays team information, and includes a live log monitor section.

The website is also connected to the logging pipeline. This means it does not only display project information; it also generates logs when users interact with it. The uploaded website file contains both the front-end interface and the JavaScript event logging logic used to send website activity into AWS.

This stage is important because it provides a realistic live source of user activity logs. Instead of relying only on static test files, the system can create new logs based on real website actions.

Stage 2: Website Event Logging

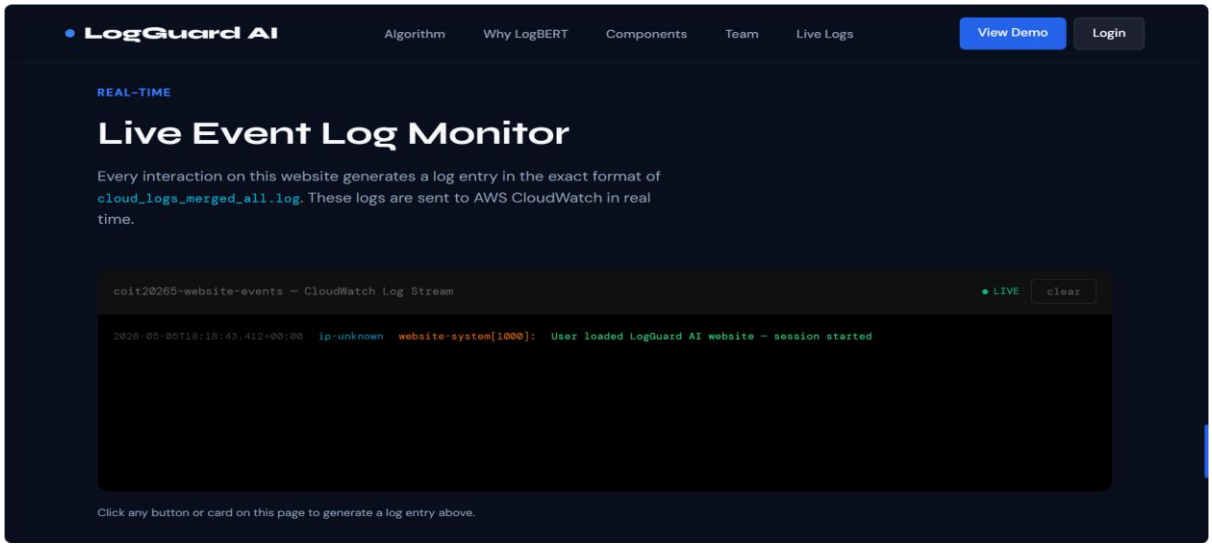


Figure 3: Website live log monitor section

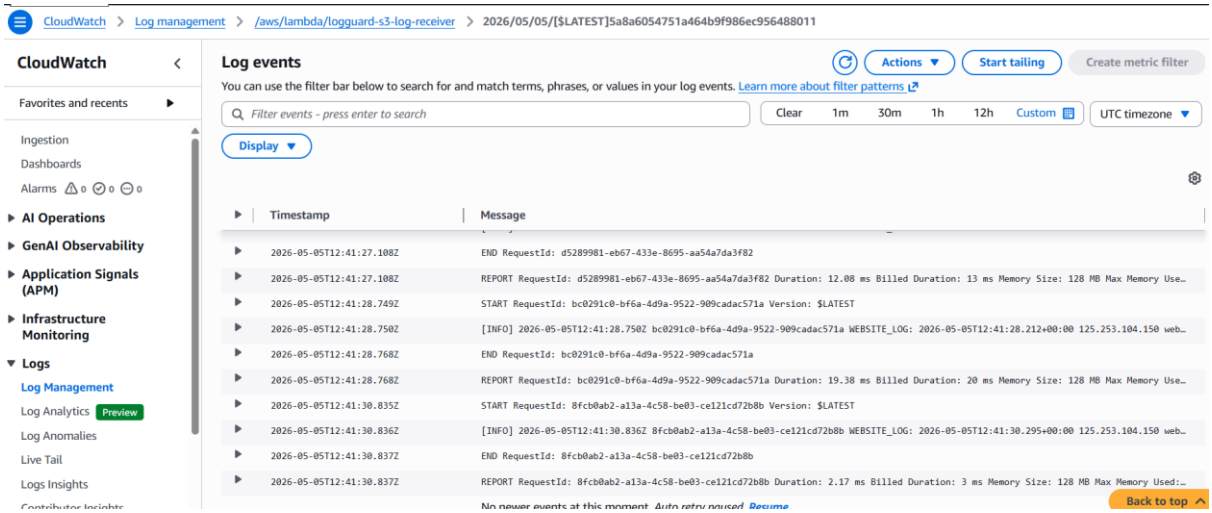


Figure 4: Website interaction logs appearing in CloudWatch

The website includes a built-in logging mechanism. Whenever a user interacts with important parts of the website, a log event is generated.

Examples of logged actions include page loading, navigation clicks, button clicks, card clicks, login attempts, and live monitor actions.

Each event is converted into a structured log message. The message contains useful details such as the event time, visitor information, website component, type of action, and a short explanation of what happened.

This makes the website activity look similar to real application logs. For example, a login failure or suspicious interaction can be represented as a log entry that can later be analysed by the anomaly detection model.

This stage demonstrates that the front-end website can behave like a live application log source.

Stage 3: API Gateway and Lambda Log Receiver

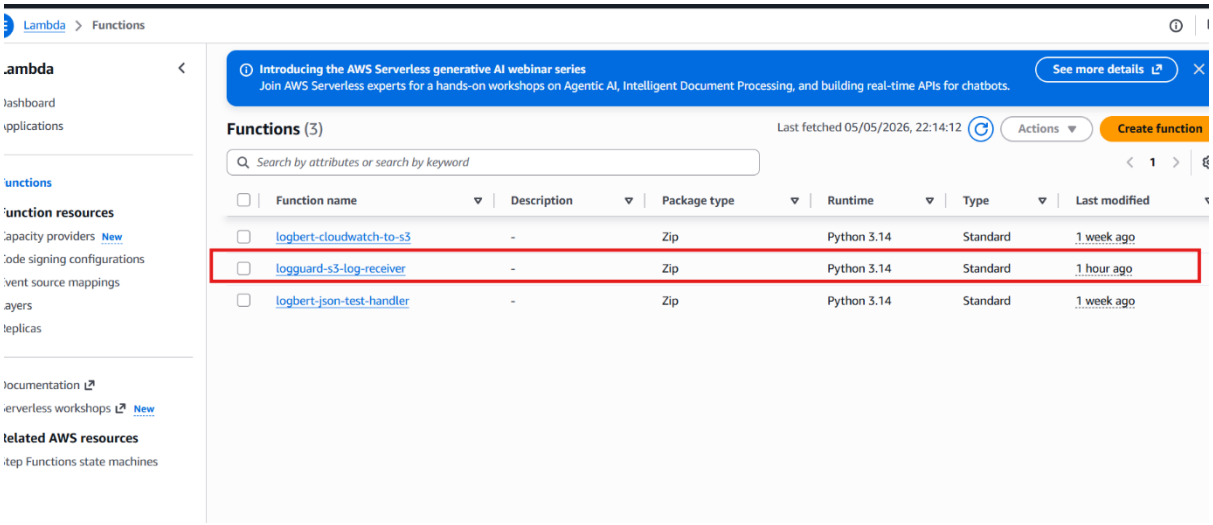


Figure 5: Lambda function list showing the website log receiver

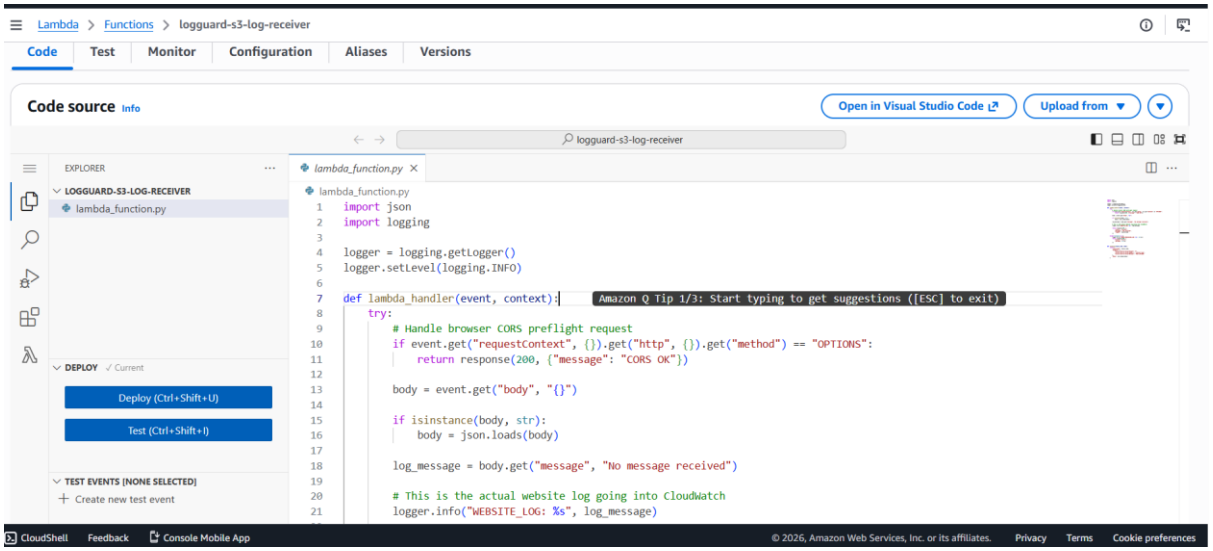


Figure 6: Lambda code screen for the website log receiver

The website does not send logs directly to CloudWatch. Instead, it sends the generated log messages through API Gateway.

API Gateway acts as the public receiving point for the website. When the website sends a log message, API Gateway triggers a Lambda function. This Lambda function receives the log, processes it, and writes it into CloudWatch.

This is a secure and practical design because the browser does not need direct AWS credentials. The website only sends a normal web request, while AWS handles the backend logging process.

The Lambda function also handles browser cross-origin requirements so that the S3-hosted website can successfully communicate with the AWS backend. The uploaded Lambda code shows that the function receives the message from the website, handles request processing, and writes the website log into CloudWatch.

Stage 4: Website Logs Stored in CloudWatch

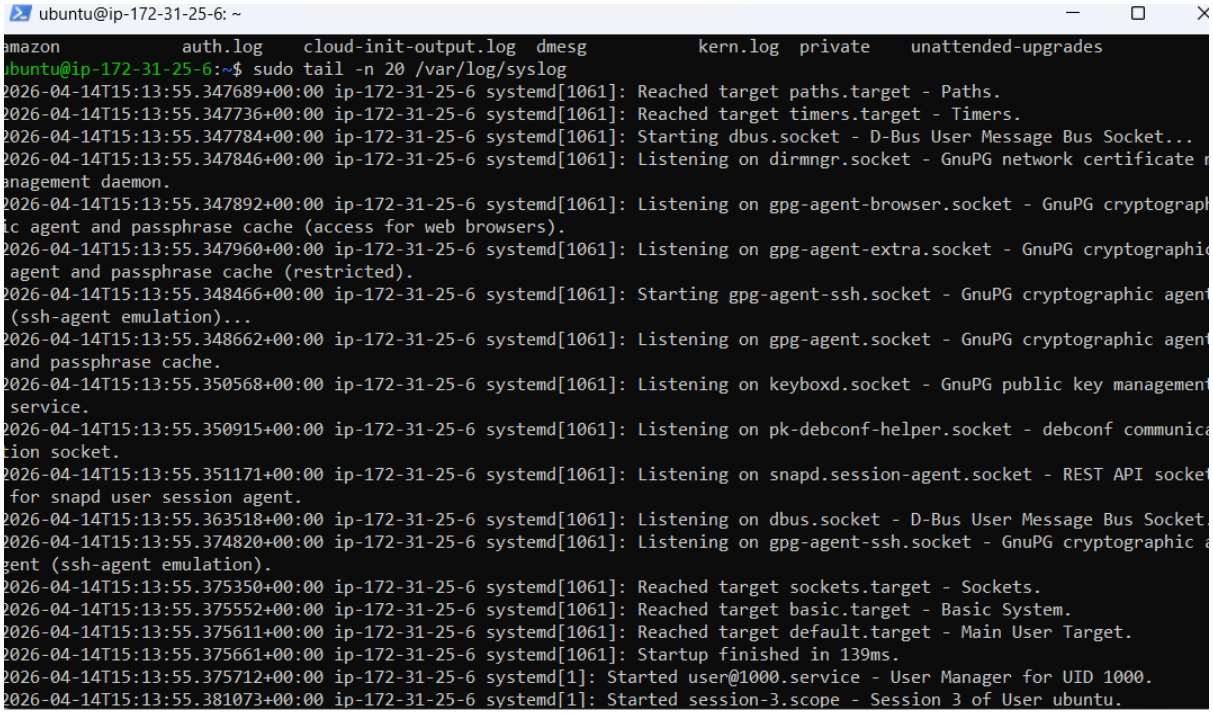


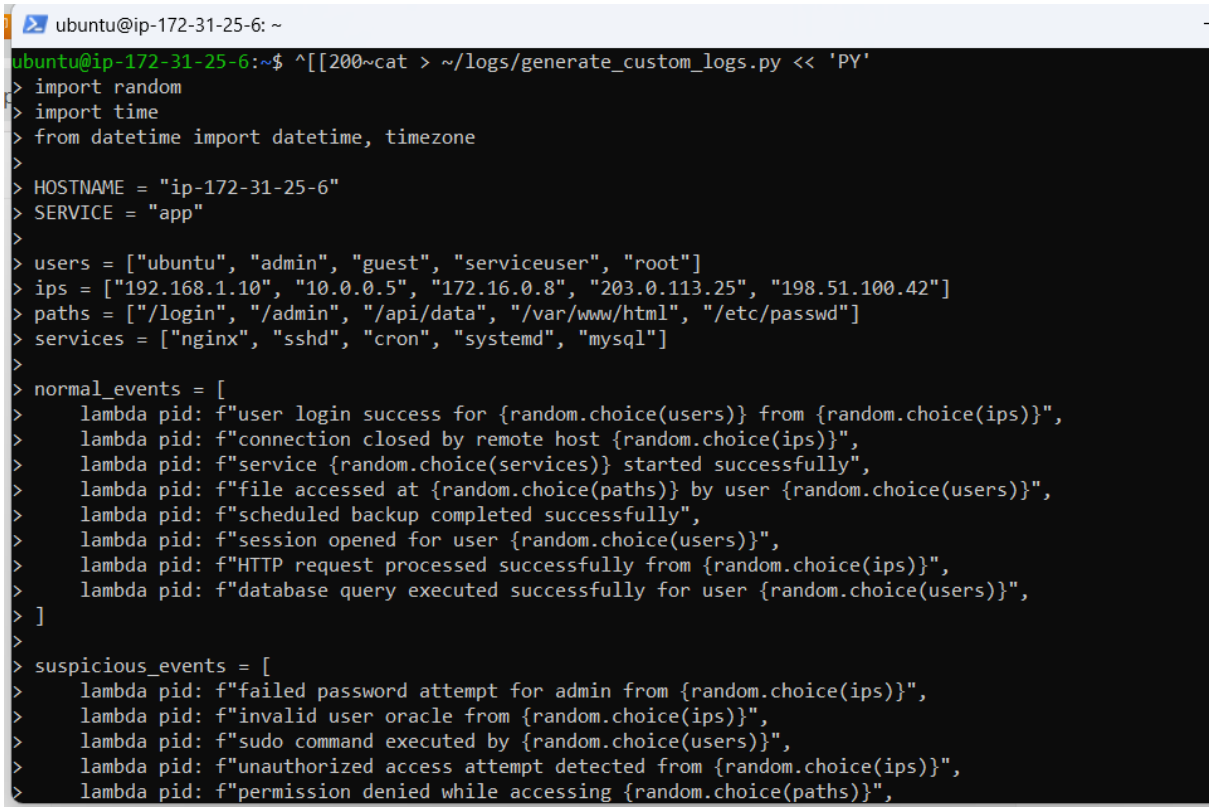
Figure 7: System logs stored and monitored, representing raw log collection before analysis

After the Lambda function receives the website log, the log is stored in Amazon CloudWatch Logs. CloudWatch becomes the central monitoring location for website activity.

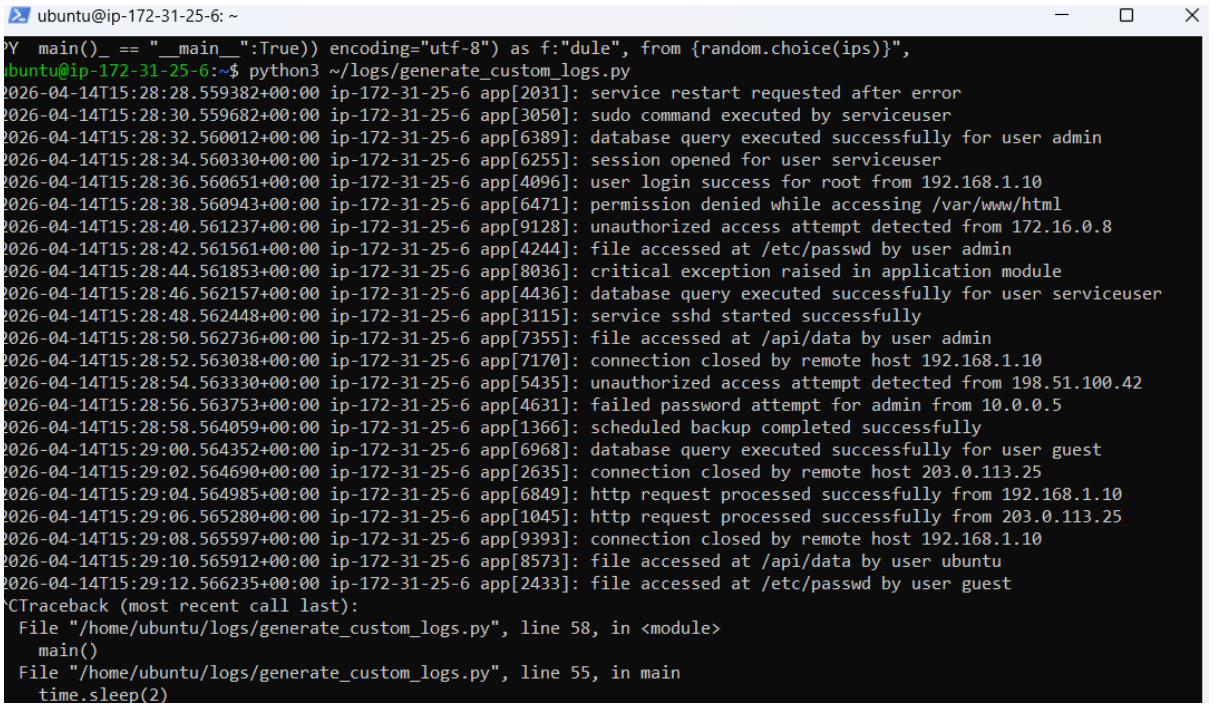
The CloudWatch log stream contains both AWS-generated Lambda execution messages and the custom website logs. The custom website logs are the important ones because they contain the actual user activity from the website.

This stage confirms that the website, API Gateway, Lambda, and CloudWatch connection is working correctly. It proves that live user interactions from the S3-hosted website are successfully reaching CloudWatch.

Stage 5: EC2 LogBERT Processing Environment



```
ubuntu@ip-172-31-25-6: ~
ubuntu@ip-172-31-25-6:~$ ^[[200~cat > ~/logs/generate_custom_logs.py << 'PY'
> import random
> import time
> from datetime import datetime, timezone
>
> HOSTNAME = "ip-172-31-25-6"
> SERVICE = "app"
>
> users = ["ubuntu", "admin", "guest", "serviceuser", "root"]
> ips = ["192.168.1.10", "10.0.0.5", "172.16.0.8", "203.0.113.25", "198.51.100.42"]
> paths = ["/login", "/admin", "/api/data", "/var/www/html", "/etc/passwd"]
> services = ["nginx", "sshd", "cron", "systemd", "mysql"]
>
> normal_events = [
>     lambda pid: f"user login success for {random.choice(users)} from {random.choice(ips)}",
>     lambda pid: f"connection closed by remote host {random.choice(ips)}",
>     lambda pid: f"service {random.choice(services)} started successfully",
>     lambda pid: f"file accessed at {random.choice(paths)} by user {random.choice(users)}",
>     lambda pid: f"scheduled backup completed successfully",
>     lambda pid: f"session opened for user {random.choice(users)}",
>     lambda pid: f"HTTP request processed successfully from {random.choice(ips)}",
>     lambda pid: f"database query executed successfully for user {random.choice(users)}",
> ]
>
> suspicious_events = [
>     lambda pid: f"failed password attempt for admin from {random.choice(ips)}",
>     lambda pid: f"invalid user oracle from {random.choice(ips)}",
>     lambda pid: f"sudo command executed by {random.choice(users)}",
>     lambda pid: f"unauthorized access attempt detected from {random.choice(ips)}",
>     lambda pid: f"permission denied while accessing {random.choice(paths)}",
```



```
ubuntu@ip-172-31-25-6: ~
PY  main()_ == "__main__":True)) encoding="utf-8") as f:"dule", from {random.choice(ips)}",
ubuntu@ip-172-31-25-6:~$ python3 ~/logs/generate_custom_logs.py
2026-04-14T15:28:28.559382+00:00 ip-172-31-25-6 app[2031]: service restart requested after error
2026-04-14T15:28:30.559682+00:00 ip-172-31-25-6 app[3050]: sudo command executed by serviceuser
2026-04-14T15:28:32.560012+00:00 ip-172-31-25-6 app[6389]: database query executed successfully for user admin
2026-04-14T15:28:34.560330+00:00 ip-172-31-25-6 app[6255]: session opened for user serviceuser
2026-04-14T15:28:36.560651+00:00 ip-172-31-25-6 app[4096]: user login success for root from 192.168.1.10
2026-04-14T15:28:38.560943+00:00 ip-172-31-25-6 app[6471]: permission denied while accessing /var/www/html
2026-04-14T15:28:40.561237+00:00 ip-172-31-25-6 app[9128]: unauthorized access attempt detected from 172.16.0.8
2026-04-14T15:28:42.561561+00:00 ip-172-31-25-6 app[4244]: file accessed at /etc/passwd by user admin
2026-04-14T15:28:44.561853+00:00 ip-172-31-25-6 app[8036]: critical exception raised in application module
2026-04-14T15:28:46.562157+00:00 ip-172-31-25-6 app[4436]: database query executed successfully for user serviceuser
2026-04-14T15:28:48.562448+00:00 ip-172-31-25-6 app[3115]: service sshd started successfully
2026-04-14T15:28:50.562736+00:00 ip-172-31-25-6 app[7355]: file accessed at /api/data by user admin
2026-04-14T15:28:52.563038+00:00 ip-172-31-25-6 app[7170]: connection closed by remote host 192.168.1.10
2026-04-14T15:28:54.563330+00:00 ip-172-31-25-6 app[5435]: unauthorized access attempt detected from 198.51.100.42
2026-04-14T15:28:56.563753+00:00 ip-172-31-25-6 app[4631]: failed password attempt for admin from 10.0.0.5
2026-04-14T15:28:58.564059+00:00 ip-172-31-25-6 app[1366]: scheduled backup completed successfully
2026-04-14T15:29:00.564352+00:00 ip-172-31-25-6 app[6968]: database query executed successfully for user guest
2026-04-14T15:29:02.564690+00:00 ip-172-31-25-6 app[2635]: connection closed by remote host 203.0.113.25
2026-04-14T15:29:04.564985+00:00 ip-172-31-25-6 app[6849]: http request processed successfully from 192.168.1.10
2026-04-14T15:29:06.565280+00:00 ip-172-31-25-6 app[1045]: http request processed successfully from 203.0.113.25
2026-04-14T15:29:08.565597+00:00 ip-172-31-25-6 app[9393]: connection closed by remote host 192.168.1.10
2026-04-14T15:29:10.565912+00:00 ip-172-31-25-6 app[8573]: file accessed at /api/data by user ubuntu
2026-04-14T15:29:12.566235+00:00 ip-172-31-25-6 app[2433]: file accessed at /etc/passwd by user guest
^CTraceback (most recent call last):
  File "/home/ubuntu/logs/generate_custom_logs.py", line 58, in <module>
    main()
  File "/home/ubuntu/logs/generate_custom_logs.py", line 55, in main
    time.sleep(2)
```

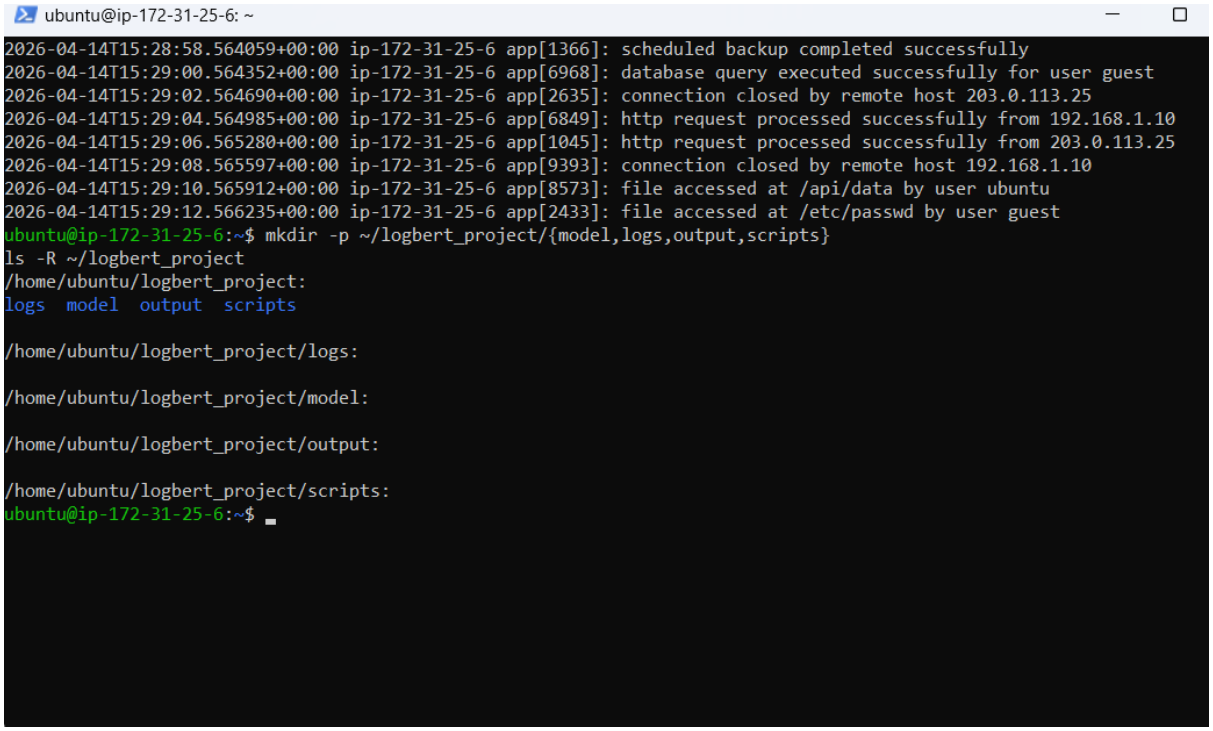


Figure 8: EC2 environment setup and custom log generation process for LogBERT analysis

The LogBERT model currently runs inside an EC2 instance. This EC2 server is used as the processing environment for anomaly detection.

At the moment, the EC2 setup uses locally generated test logs. A log generator creates a mixture of normal and suspicious log events, such as successful logins, file access, database activity, failed password attempts, unauthorised access attempts, permission denied messages, and service errors.

The LogBERT checker reads these logs and analyses them in grouped windows. For each group of logs, the model calculates an anomaly score. It then identifies the alert level, alert type, possible reason, and sample logs related to the suspicious behaviour.

The terminal output shows that LogBERT is successfully producing anomaly results. This confirms that the model is working on the EC2 server. The uploaded EC2 command notes also show the current workflow for generating logs, running LogBERT, and checking the anomaly output.

Stage 6: LogBERT Anomaly Results Stored as Structured JSON

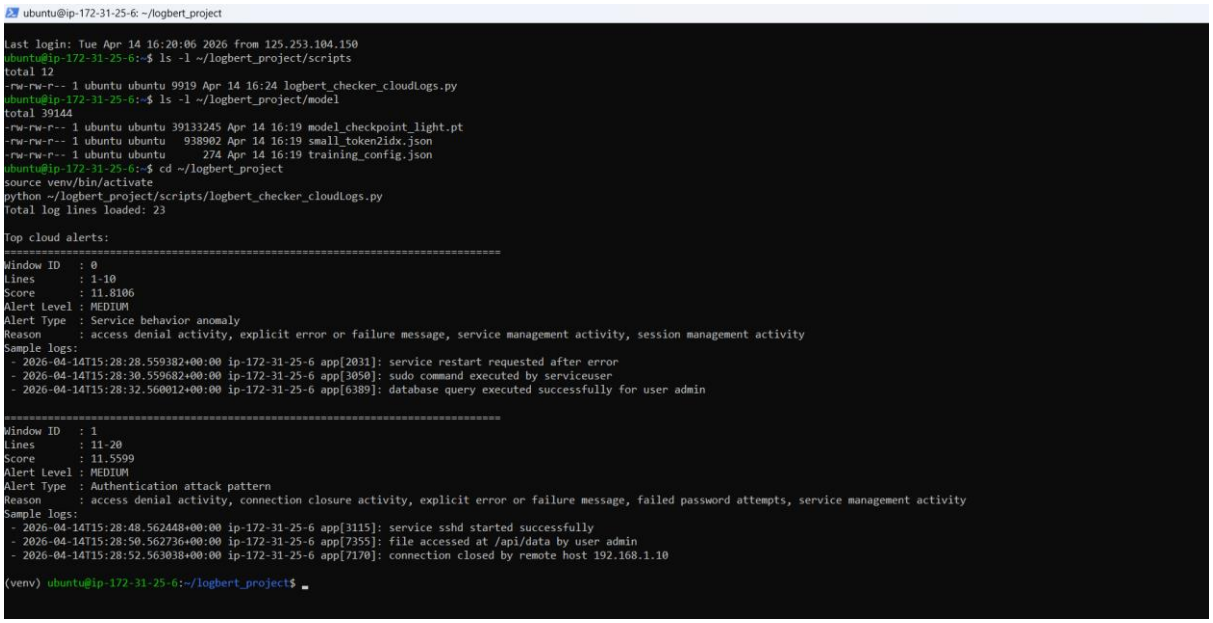


Figure 9: LogBERT anomaly detection output showing structured alert results including score, alert level, and reasoning

After LogBERT detects suspicious log windows, the results are saved as structured JSON alerts in CloudWatch.

Each anomaly result contains important details such as the model source, window number, affected log lines, anomaly score, alert level, alert type, reason for detection, and sample logs.

This structured format is important because the result can be reused by other parts of the system. The same anomaly alert can be used for email notification, AI-generated recommendation, S3 storage, and dashboard visualisation.

This stage turns the model output into a proper cloud-based alert object instead of keeping it only as terminal text.

Stage 7: SNS Email Alerting with OpenAI Recommendations

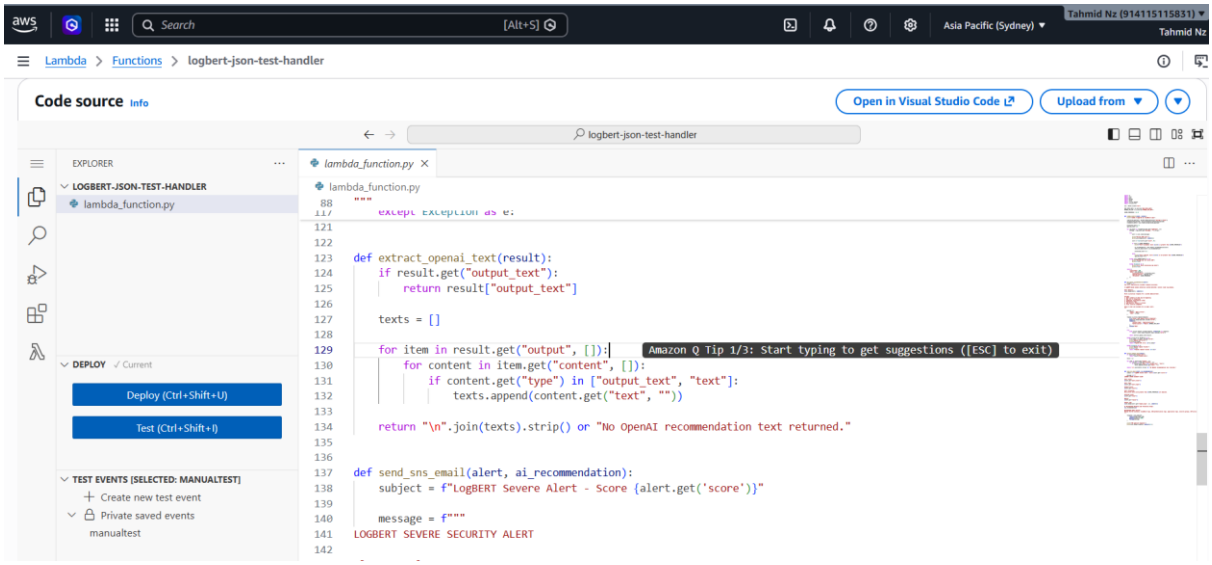


Figure 10: AWS Lambda function integrating LogBERT results with SNS and OpenAI to generate automated alert notifications

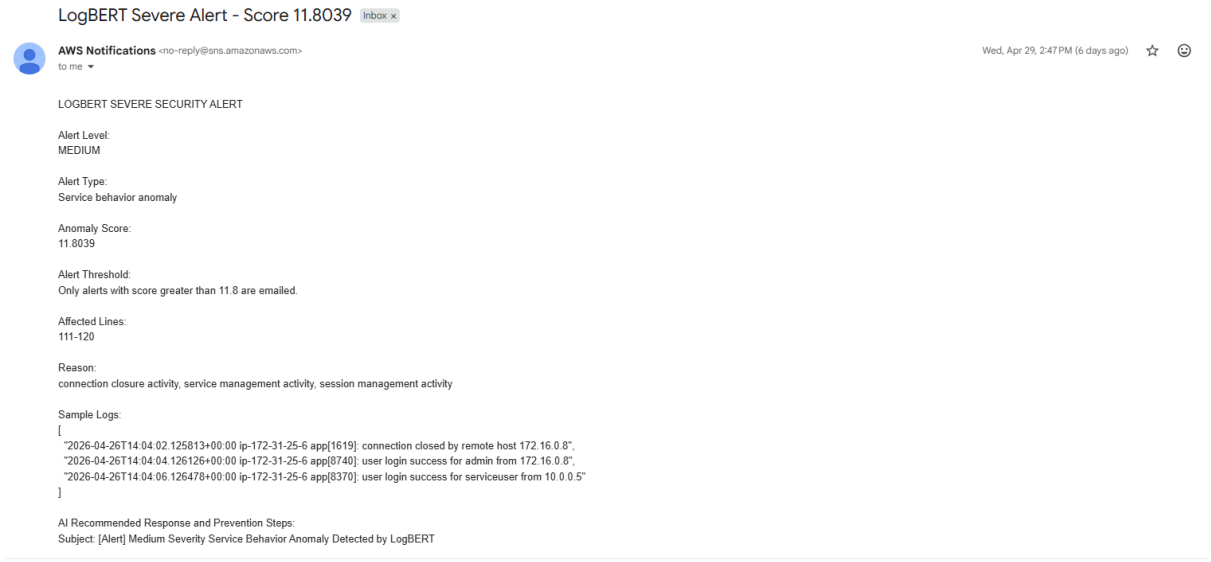


Figure 11: Email alert generated via SNS including anomaly details and AI-based recommended response actions

The project includes an automated email alerting stage. When a LogBERT anomaly score passes the selected threshold, a Lambda function prepares an alert email and sends it using Amazon SNS.

Before sending the email, the Lambda function sends the anomaly context to OpenAI. OpenAI generates recommended response and prevention steps based on the alert details. These recommendations are then included in the final email.

The email alert contains information such as the alert level, alert type, anomaly score, affected log lines, reason for detection, sample logs, and AI-generated response guidance.

This stage is important because it moves the project beyond basic detection. The system does not only identify suspicious behaviour; it also helps explain what actions should be taken after the anomaly is detected.

The working flow is that LogBERT produces an anomaly result, the alerting Lambda checks the score, OpenAI generates security response guidance, and SNS sends the final alert email to the user.

Stage 8: CloudWatch to S3 Storage for Dashboard

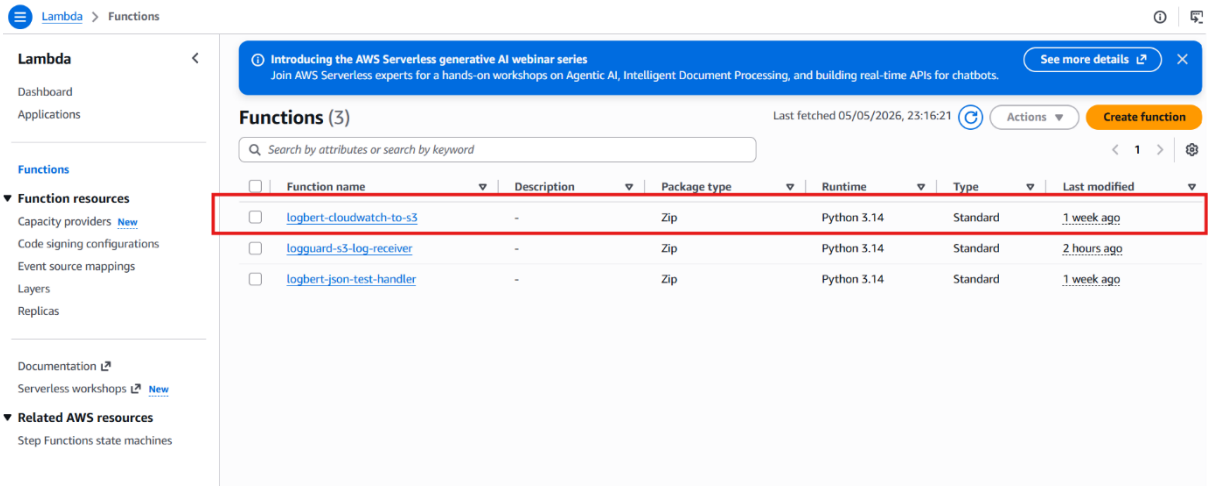


Figure 12: Lambda function used to move CloudWatch alerts to S3

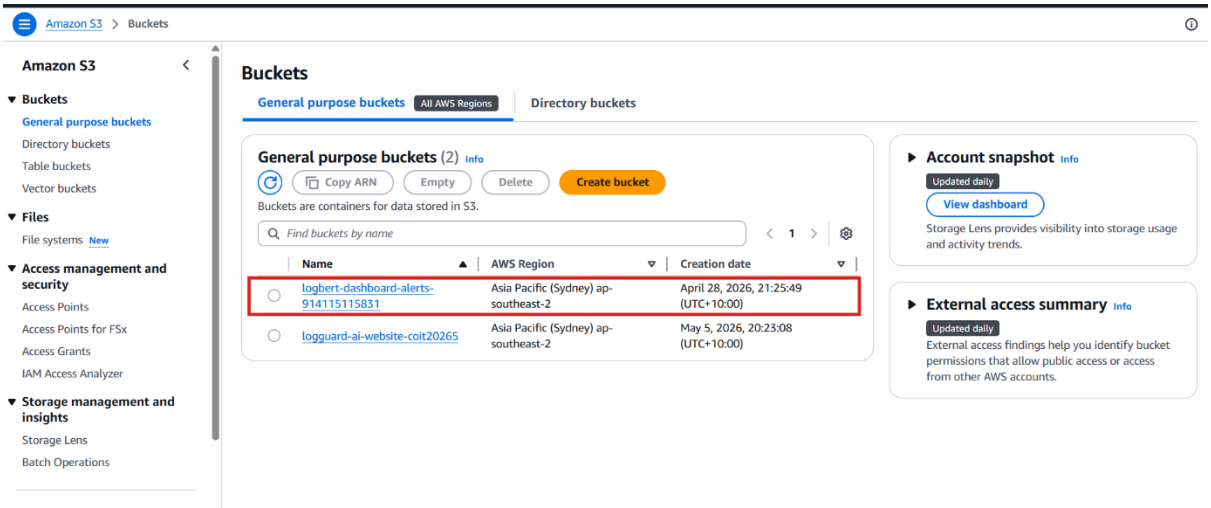


Figure 13: S3 bucket used for dashboard alert storage

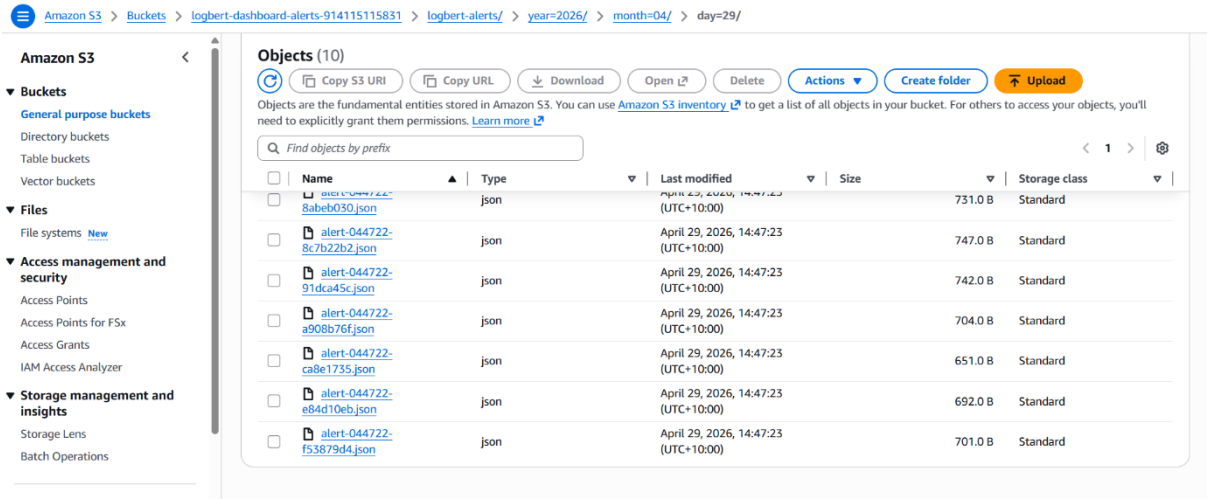


Figure 14: S3 folder showing stored alert JSON files

The dashboard needs a clean and reliable data source. For this reason, the LogBERT anomaly JSON alerts are moved from CloudWatch into Amazon S3.

A Lambda function is used for this stage. It receives CloudWatch anomaly events, extracts valid JSON alert data, and stores each alert as a separate JSON file in S3. Any non-JSON messages are skipped so that only valid alert data is stored.

The alerts are organised in S3 using a date-based structure. This makes the alert data easier to manage, search, and load into the dashboard.

This stage is important because it separates the dashboard from the model execution process. The dashboard does not need to depend directly on the EC2 terminal or the model runtime. It can simply read the stored alert files from S3.

The uploaded dashboard Lambda code shows that the function decodes CloudWatch log data, parses valid JSON alerts, and saves them into the dashboard S3 storage location.

Stage 9: Streamlit Dashboard Visualisation

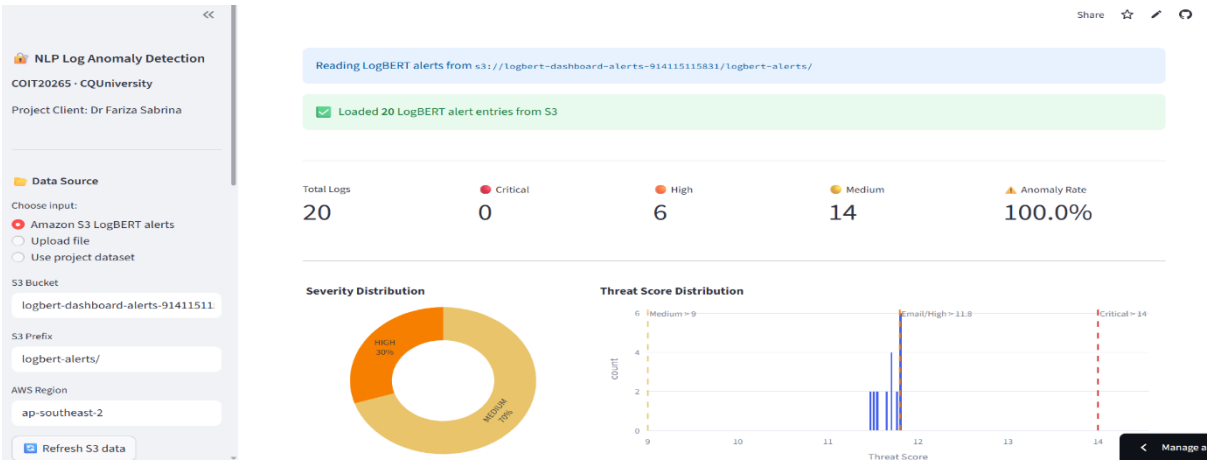


Figure 15: Interactive dashboard visualising LogBERT anomaly alerts retrieved from S3, including severity distribution and threat score analysis

The dashboard provides a visual representation of anomaly detection results generated by LogBERT. It retrieves structured JSON alerts stored in Amazon S3 and displays key insights such as total log entries, severity levels, and anomaly rates.

Additionally, graphical components including a severity distribution chart and threat score histogram allow for easier interpretation of anomaly patterns. This enables quick identification of potential threats and supports decision-making for security monitoring.

Further Improvements and Planned Upgrades

1. Upgrade EC2 to Use Live CloudWatch Logs

The EC2 stage currently uses generated local logs for testing. The next improvement is to make EC2 fetch live logs from CloudWatch.

This will allow LogBERT to analyse real website activity logs instead of only test logs. The EC2 process should collect new CloudWatch log entries, clean them, and pass only useful log messages into the LogBERT model.

This upgrade will complete the live connection between the website logging system and the anomaly detection model.

2. Improve Email Thread Organisation

The current email alerting system sends alert emails when scores pass the selected threshold. In the future, the email thread should be organised more clearly.

Planned improvements include:

1. Add a unique incident ID to each alert.
2. Use clearer subject lines with severity and alert type.
3. Group similar alerts where possible.
4. Reduce duplicate emails for repeated alerts.
5. Make the email easier to read during incident review.

This will make the alerting system more professional and easier to manage.

3. Add NIST Framework-Based Recommendations

The AI-generated email recommendations should be improved using the NIST cybersecurity framework. Instead of giving general advice, the email can organise response steps under clear security categories.

The recommended structure should include:

1. Govern
2. Identify
3. Protect
4. Detect
5. Respond
6. Recover

This will make the alert email more useful because it will guide the user through a structured security response process. It will also make the project stronger from a cybersecurity governance and incident response perspective.

Final Target System

The final upgraded system will connect the website logs directly to the LogBERT model through CloudWatch and EC2.

Once this is completed, the full system will work as a live cloud-based anomaly detection pipeline: website activity will be logged, stored in CloudWatch, fetched by EC2, analysed by LogBERT, stored as structured alerts, sent through SNS with AI-supported recommendations, saved into S3, and visualised in the dashboard.

This final version will demonstrate a complete AI-based cybersecurity monitoring and response prototype.